



JavaScript

Web Application Development

KIEU Quoc Viet HUYNH Vinh Nam

Information and Communication Technology Laboratory (ICTLab),
University of Science and Technology of Hanoi

Hanoi, August 2026



Course Syllabus

Class	Module	Time	Type	Topic
1	Introduction to Web Application Development	4h	Lecture (3h) Practical (1h)	- Why web applications, and where they came from - Client-server model & HTTP - From page to application - Engineering in the age of AI
2	HTML5 & CSS3	4h	Lecture (3h) Practical (1h)	- Semantic structure, accessibility - Box model, Flexbox & Grid - Responsive design, media queries
3	JavaScript	4h	Lecture (3h) Practical (1h)	- ES6+ syntax, functions, objects - DOM manipulation & events - Promise, async/await - Fetch API and error handling
4	Node.js Fundamentals	4h	Lecture (3h) Practical (1h)	- Runtime, modules, the event loop - npm and the package ecosystem - Build tooling for the front-end
5	Front-end Frameworks	4h	Lecture (3h) Practical (1h)	- Components and composition - Props, state and hooks - Client-side routing - Consuming a REST API

Class 2 gave you a static Pho Thin page. Today it reacts.

Table of Contents

- 1 Setup
- 2 Modern JavaScript (ES6+)
- 3 Functions and Objects
- 4 The DOM and Events
- 5 Asynchronous JavaScript

1. Setup

Before You Write Any Code

- Nothing new to install - you already have a browser, VS Code and the Live Server extension from class 2
- Today's project is three files in one folder: `index.html`, `app.js`, `menu.json`
- The JavaScript lives in a separate `.js` file, never mixed into the HTML
- `index.html` loads it with `<script src="app.js" defer>` - `defer` makes the script run *after* the HTML is parsed, so the elements it looks for already exist

Open the folder with Live Server, not by double-clicking the file. A page opened as `file://` cannot fetch `menu.json` - the browser blocks it.

Watch out All three files go in the same folder. A wrong path is the number one reason "my code does nothing" on day one.

The Standard Page

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pho Thin</title>
</head>
<body>
  <h1 id="heading">Pho Thin</h1>
  <ul id="menu">
    <li class="dish">Pho bo</li>
    <li class="dish">Pho ga</li>
    <li class="dish">Pho tai</li>
  </ul>
  <form id="signup">
    <input id="customer"
      placeholder="Your name">
    <button>Order</button>
  </form>
  <div id="out"></div>
  <script src="app.js" defer></script>
</body>
</html>
```

app.js

```
// ===== PASTE HERE =====

// =====
```

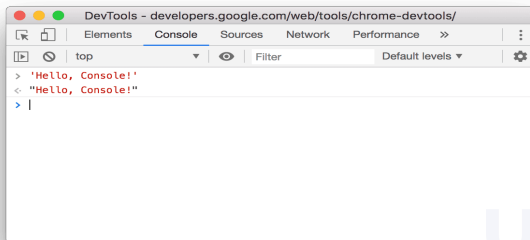
Three rules for today

- index.html is typed once and never changes
- One snippet at a time - delete the old one first
- Save, then reload the browser

Two Places to Run JavaScript

	The Console	The file app.js
How to open	F12, then the Console tab	VS Code
Good for	trying one line, checking a value	the real code of your app
Survives a reload	No	Yes
Shows results	Types the answer back at you	Only what you console.log

- Every code slide today is meant to be pasted into app.js, then read in the Console
- Typing `2 + 2` in the console prints 4 straight away. The same line in app.js prints nothing - a file only shows what you ask it to show



When It Breaks

- A red line in the Console is JavaScript telling you exactly what went wrong - read it, do not guess
- Chrome prints the message, then the file and line number, like `app.js:14`. Click it and the editor jumps there
- After the first red error the rest of the file does not run. Fix the top one first

What the Console says	What it usually means
Uncaught SyntaxError: Unexpected token	A missing <code>)</code> <code>}</code> or <code>;</code> - look at the line above too
Uncaught ReferenceError: <code>x</code> is not defined	You used a name you never declared, or spelled it differently
Cannot read properties of null	<code>querySelector</code> found nothing - wrong selector, or the script ran before the HTML
Nothing at all appears	The file is not loaded (wrong <code>src</code>), or your code has no <code>console.log</code>

2. Modern JavaScript

What JavaScript Is

- The programming language of the web - it runs in every browser
- High-level and multi-paradigm: imperative, functional and object-oriented
- Dynamically typed and just-in-time compiled - fast, and nothing to compile before you run it
- No longer only in the browser: with **Node.js** the same language runs on servers, tools and desktop apps (class 4)
- JavaScript is the language; **ECMAScript (ES)** is the specification it follows
- **ES6 (2015)** was the turning point - `let`, `const`, arrow functions, classes, promises, modules
- Since then one small release every year - ES2016, ES2017, ... up to ES2026
- The name has nothing to do with Java - it was a 1995 marketing decision

Source: ecma-international.org - ECMA-262, 17th edition, June 2026

The Three Languages of the Front-End

One page, three jobs

HTML

structure
what is on the page

CSS

presentation
how it looks

JavaScript

behaviour
what it does

- HTML and CSS **describe**; only JavaScript can **react and decide**
- It handles clicks and typing, validates forms, and fetches fresh data from a server without reloading the page
- Turning a static **page** into an interactive **application** is exactly what JavaScript adds

What Carries Over From Your First Course

Concept	You already know (C / Python)	In JavaScript
Variables	A named box for a value	Same idea - <code>let</code> / <code>const</code>
Types	Written down, fixed	Decided at run time (dynamic)
<code>if</code> / <code>else</code> , loops	You have used these	Same, plus <code>for...of</code>
Functions	Take input, return output	Same, plus arrow functions
Arrays / lists	Indexed from 0	Same, plus <code>map</code> / <code>filter</code>
Objects	You have studied OOP	Same idea, written as <code>{ key: value }</code>

The logic is the same. What is new is the browser, the DOM, and talking to a server - the last three sections today.

Variables and Types

- `const` by default; use `let` only when the value must change
- Both are **block-scoped** - they exist only inside the `{ }` they are declared in
- `var` is function-scoped and error-prone; modern code does not use it
- Five types you meet every day: `number`, `string`, `boolean`, `null`, `undefined` (also `bigint` and `symbol`, rarely needed)
- Integers and decimals are **one type**: `number`
- `typeof` tells you the type of any value at run time
- `null = "empty on purpose"; undefined = "no value assigned yet"`

Example: Variables and Types

Recall `const` by default, `let` only when the value must change - and `typeof` tells you the type of any value at run time.

app.js

```
let count = 10;
count = count + 1;
console.log("count:", count);

const PI = 3.14159;
console.log("types:", typeof PI,
           typeof "Nam", typeof true);

let city      = "Hanoi";
let isOpen    = true;
let nothing   = null;
let notSet;
// null and undefined are NOT the same
console.log("null vs notSet:",
           typeof nothing, typeof notSet);

PI = 3;    // keep this line LAST
```

Console

```
count: 11
types: number string boolean
null vs notSet: object undefined
```

Uncaught TypeError: Assignment
to constant variable.

Watch out Anything after a red error
does not run. That is why `PI = 3` is
the last line.

Strings and Operators

- Three ways to quote a string: `'...'`, `"..."` and backticks ``...``
- Only backticks allow `${ ... }` to drop a value straight into the text, and only backticks can span several lines
- Arithmetic: `+` `-` `*` `/` `%` `**`. With two strings, `+` joins them instead of adding
- Comparison: always use `===` and `!==` (strict - value *and* type)
- Logical: `&&` (and), `||` (or), `!` (not)

Watch out `==` converts types behind your back: `5 == "5"` and `0 == ""` are both true. This is a classic beginner bug - never use it.

Example: Template Literals

Recall Only backticks allow `${ ... }` to drop a value straight into the text.

app.js

```
let city = "Hanoi";
let year = 2026;

// old way: glue with +
let a = "Welcome to " + city + " in " + year;

// modern way: backticks
let b = `Welcome to ${city} in ${year}`;

console.log("with +:", a);
console.log("with backticks:", b);
console.log("same result?", a === b);
```

Console

```
with +: Welcome to Hanoi
      in 2026
with backticks: Welcome to
      Hanoi in 2026
same result? true
```

console.log of a string prints it without quotes. Typing the same value at the console prompt shows 'quotes' instead.

Example: Operators and Strict Equality

Recall Always use `===` and `!==`, because `==` converts the types behind your back first.

app.js

```
// arithmetic
console.log("5 + 2 =", 5 + 2);
console.log('"a" + "b" =', "a" + "b");
console.log("7 % 2 =", 7 % 2);

// === compares value AND type
console.log('5 === 5 ->', 5 === 5);
console.log('5 === "5" ->', 5 === "5");
// == converts first, then compares
console.log('5 == "5" ->', 5 == "5");
console.log('0 == "" ->', 0 == "");

// logical
console.log("true && false ->", true && false);
console.log("true || false ->", true || false);
console.log("!true ->", !true);
```

Console

```
5 + 2 = 7
"a" + "b" = ab
7 % 2 = 1
5 === 5 -> true
5 === "5" -> false
5 == "5" -> true
0 == "" -> true
true && false -> false
true || false -> true
!true -> false
```

Watch out `Number("")` is 0, not NaN - an empty string, or one of only spaces, converts to zero. That is why `0 == ""` is true.

Making Decisions and Repeating

- `if / else` works exactly as in C or Python
- The **ternary** condition `? ifTrue : ifFalse` is a compact `if/else` that *returns a value*
- Exactly **nine** values are **falsey**: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`, and the legacy `document.all`. Everything else is **truthy**
- `switch` is good when one value is compared against many fixed options - every case needs `break` or execution "falls through" to the next
- Loops: classic `for` when you need the index, `for...of` to walk a list, `while` to repeat while a condition holds

Prefer `for...of` for arrays - it is shorter and avoids off-by-one index mistakes.

Example: if / else and the Ternary

Recall The ternary condition ? ifTrue : ifFalse returns a value, and exactly nine values are falsy.

app.js

```
let grade = 72;

if (grade >= 60) {
  console.log("if/else ->", "Passed");
} else {
  console.log("if/else ->", "Failed");
}

let result = grade >= 60 ? "Passed" : "Failed";
console.log("ternary ->", result);

// the falsy values, one by one
console.log('0, empty, "0" ->', Boolean(0),
  Boolean(""), Boolean("0"));
console.log("false, -0, 0n ->", Boolean(false),
  Boolean(-0), Boolean(0n));
```

Console

```
if/else -> Passed
ternary -> Passed
0, empty, "0" -> false false
true
false, -0, 0n -> false false
false
```

"0" is a **non-empty string**, so it is
truthy - even though the number
0 is falsy.

Example: switch

Recall switch compares one value against many fixed options, and every case needs break or it falls through.

app.js

```
let code = 84;
let country;

switch (code) {
  case 84:
    country = "Vietnam";
    break;
  case 1:
    country = "US";
    break;
  default:
    country = "Unknown";
}
console.log("code 84 ->", country);
```

Console

```
// as written above
code 84 -> Vietnam

// now delete the first break
// and reload
code 84 -> US
```

Watch out Without break, every case below also runs - and the last one wins.

Example: 3 kinds of Loops

Recall Classic for when you need the index, for...of to walk a list, while to repeat while a condition holds.

app.js

```
// classic for - when you need the index
for (let i = 0; i < 3; i++) {
  console.log("for i =", i);
}

// for...of - walk through a list
const cities = ["Hanoi", "Hue", "Da Nang"];
for (const city of cities) {
  console.log("for...of city =", city);
}

// while - repeat while a condition holds
let n = 1;
while (n <= 3) {
  n++;
}
console.log("while ended, n =", n);
```

Console

```
for i = 0
for i = 1
for i = 2
for...of city = Hanoi
for...of city = Hue
for...of city = Da Nang
while ended, n = 4
```

3. Functions and Objects

Functions

- A function packages code you can call again with different inputs
- `return` sends a value back and ends the function
- A default parameter fills in a value when the caller gives none: `function greet(who = "friend")`
- Arrow functions are a shorter syntax introduced in ES6
- One parameter: the parentheses are optional. One expression: the `return` and the braces are optional
- They are everywhere in modern code - especially as arguments to array methods and event handlers, which is the next two sections

Defining a function runs nothing. Nothing happens until you call it - a snippet that only defines functions prints an empty console.

Example: Functions

Recall `return` sends a value back and ends the function, and a default parameter fills in when the caller gives none.

app.js

```
function areaOfCircle(r) {  
  return 3.14159 * r * r;  
}  
  
const greet = function (name = "guest") {  
  return "Hello, " + name;  
};  
  
function noReturn() { }  
  
// nothing runs until you CALL them  
console.log("area r=2:", areaOfCircle(2));  
console.log("with a name:", greet("Nam"));  
console.log("with default:", greet());  
console.log("no return:", noReturn());
```

Console

```
area r=2: 12.56636  
with a name: Hello, Nam  
with default: Hello, guest  
no return: undefined
```


Example: Arrow Functions

Recall With one parameter the parentheses are optional, and with one expression the return and the braces are optional too.

app.js

```
// these three do exactly the same thing
const square1 = function (x) {
  return x * x;
};
const square2 = (x) => { return x * x; };
const square3 = x => x * x;

console.log("all three squares:",
  square1(4), square2(4), square3(4));

const add = (a, b) => a + b;
console.log("add(2, 3):", add(2, 3));
```

Console

```
all three squares: 16 16 16
add(2, 3): 5
```

Passing several arguments to one `console.log` prints them on one line, separated by a space.

Objects and Arrays

- An object groups related data (properties) and behaviour (methods) under named keys
- Read a property with a dot (`student.year`) or with brackets (`student["year"]`)
- This is exactly the shape of the JSON you will receive from a server in the last section
- Destructuring unpacks an object or an array in one line
- Spread (...) copies or merges without touching the original
- `map`, `filter` and `reduce` transform an array **without a loop** - each one takes a function

`map` returns a new array of the same length, `filter` a shorter one, `reduce` a single value.

Example: Objects

Recall An object groups properties and methods under named keys, read with a dot or with brackets.

app.js

```
const dish = {  
  name: "Pho bo",  
  price: 60000,  
  spicy: false,  
  label() {  
    return `${this.name} costs ${this.price}`;  
  }  
};  
  
console.log("dot access:", dish.name);  
console.log("bracket access:", dish["price"]);  
console.log("method:", dish.label());  
// a key that does not exist is not an error  
console.log("missing key:", dish.sauce);
```

Console

```
dot access: Pho bo  
bracket access: 60000  
method: Pho bo costs 60000  
missing key: undefined
```

const stops you replacing the whole object, but you can still add and change its properties.

Example: Destructuring and Spread

Recall Destructuring unpacks an object or an array in one line, and spread ... copies or merges without touching the original.

app.js

```
const dish = { title: "Pho bo", price: 60000 };
const scores = [8, 9, 10];

// destructuring: pull values out
const { title, price } = dish;
const [first, second] = scores;
// title and price are now normal variables
console.log("from object:", title, price);
console.log("from array:", first, second);

// spread: copy or merge
const copy = { ...dish, sold: true };
const all = [...scores, 7];
console.log("merged copy:", copy);
console.log("extended array:", all);
```

Console

```
from object: Pho bo 60000
from array: 8 9
merged copy:
  {title: 'Pho bo', price: 60000,
   sold: true}
extended array:
  (4) [8, 9, 10, 7]
```

Logging an object or an array shows quotes around the strings inside, and Chrome prefixes an array with its length.

Example: Array Methods: map, filter, reduce

Recall `map` returns a new array of the same length, `filter` a shorter one, and `reduce` a single value.

app.js

```
const nums = [1, 2, 3, 4];

// forEach just visits, it returns nothing
nums.forEach(n =>
  console.log("forEach saw:", n));

const doubled = nums.map(n => n * 2);
const evens    = nums.filter(n => n % 2 === 0);
const total    = nums.reduce((s, n) => s + n, 0);

console.log("map doubled:", doubled);
console.log("filter evens:", evens);
console.log("reduce total:", total);
console.log("original nums:", nums);
```

Console

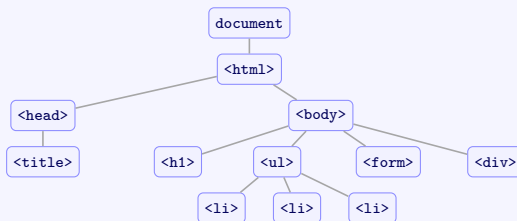
```
forEach saw: 1
forEach saw: 2
forEach saw: 3
forEach saw: 4
map doubled:   (4) [2, 4, 6, 8]
filter evens:  (2) [2, 4]
reduce total:  10
original nums: (4) [1, 2, 3, 4]
```

The last line proves it: `nums` is **unchanged**. All three methods return a new value.

4. The DOM and Events

The DOM: The Page as a Tree

The standard page, as the browser sees it



- The browser turns your HTML into the DOM - a tree of objects, one node per tag, arranged as a tree. DOM = Document Object Model
- Every tag becomes a **node** you can read and change from JavaScript, the page updates live, the user sees it immediately, with no reload
- Think of the folder tree in your file explorer: parents, children, nesting
- This is the tree of **your** `index.html` - every selector in this section points at one of these nodes

Finding and Changing Elements

- `getElementById("menu")` when the element has an id - the clearest and fastest way
- `querySelector` takes any CSS selector (`#id`, `.class`, `tag`) and returns the first match - use it when there is no id
- `querySelectorAll` returns all matches as a list you can walk with `forEach`
- `textContent` changes the text inside an element
- `classList.add.remove.toggle` switches CSS classes on and off
- `style.color` sets a single CSS property directly
- `createElement` plus `append` puts a brand-new node into the tree

Watch out Use `textContent` to set text. `innerHTML` with untrusted input lets an attacker inject markup - we come back to this in class 9.

Example: Selecting Elements

Recall `getElementById` when there is an id, `querySelector` for the first CSS match, `querySelectorAll` for all of them.

app.js

```
const titleEl = document.getElementById("heading");
const firstDish = document.querySelector(".dish");
const allDishes = document.querySelectorAll(".dish");
const menuList = document.querySelector("#menu");

console.log("by id:", titleEl.textContent);
console.log("first .dish:", firstDish.textContent);
console.log("how many .dish:", allDishes.length);
console.log("tag name:", menuList.tagName);

allDishes.forEach(d => console.log("dish:",
d.textContent));
```

Console

```
by id: Pho Thin
first .dish: Pho bo
how many .dish: 3
tag name: UL
dish: Pho bo
dish: Pho ga
dish: Pho tai
```

Example: Changing the Page

Recall `textContent` changes text, `classList` switches CSS classes, and `createElement` plus `append` puts a new node into the tree.

app.js

```
const titleEl = document.getElementById("heading");
titleEl.textContent = "Welcome to Pho Thin";
titleEl.style.color = "crimson";

const li = document.createElement("li");
li.textContent = "Pho sot vang";
li.classList.add("dish");
document.getElementById("menu").append(li);

document.getElementById("out").textContent =
  `Now ${document.querySelectorAll('.dish').length}
  dishes`;
```

What the page shows

before

Pho Thin

• Pho bo • Pho ga • Pho tai

after

**Welcome to Pho
Thin**

• Pho bo • Pho ga • Pho tai •

Pho sot vang

Now 4 dishes

Events and Forms

- `addEventListener(type, handler)` is the modern way to respond to what the user does
- It keeps behaviour in the `.js` file, not in `onclick="..."` attributes inside the HTML
- Common types: `click`, `input`, `change`, `submit`, `keydown`
- The handler receives an **event object**; `event.target` is the element that fired it and `event.target.value` is what the user typed
- `event.preventDefault()` stops the browser's default action - for a form that default is **submit and reload the page**
- Read a field with `.value`, and `.trim()` away the spaces before you check it

Watch out Never trust the client. Checking in the browser is for *convenience*; the server must check again (class 7).

Example: Events

Recall `addEventListener(type, handler)` responds to the user, and the handler receives an event whose target is the element that fired it.

app.js

```
const button = document.querySelector("#signup");
const nameBox = document.getElementById("customer");

button.addEventListener("click", (event) => {
  event.preventDefault();
  console.log("click ->", "Order button");
});

nameBox.addEventListener("input", (event) => {
  // event.target = the element that fired it
  console.log("typing ->", event.target.value);
});
```

Console

```
// you click Order
click -> Order button

// you type P, h, o
typing -> P
typing -> Ph
typing -> Pho
```

Without `preventDefault()` the button submits the form and the page reloads - the console clears before you can read it.

Example: Forms and Validation

Recall `preventDefault()` stops the form's submit-and-reload, and `.value` with `.trim()` reads the field.

app.js

```
const form    = document.getElementById("signup");
const nameBox = document.getElementById("customer");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const customer = nameBox.value.trim();
  if (customer === "") {
    alert("Name must be filled out");
    return;
  }
  console.log("submitted for:", customer);
});
```

What happens

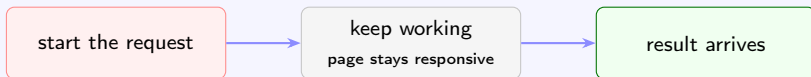
```
// empty box, press Order
a dialog appears:
Name must be filled out
the page does NOT reload

// type Nam, press Order
submitted for: Nam
```

5. Asynchronous JS

Why Asynchronous

Start the slow job, keep working, collect the result later



- Some tasks take time: fetching data from a server, reading a file, a timer
- JavaScript runs on a **single thread** - it must not freeze while waiting
- So slow tasks run **in the background** and notify you when they finish

Remember the pho shop from class 1? You order, get a ticket, and keep chatting - the food arrives later. A **Promise** is that ticket: a value that is *not ready yet*.

The menu.json File

menu.json - type it once, never change it

```
[  
  { "name": "Pho bo", "price": 60000 },  
  { "name": "Pho ga", "price": 55000 },  
  { "name": "Pho tai", "price": 65000 }  
]
```

JSON looks like a JavaScript array of objects, with one rule: every key is in double quotes, and there is no trailing comma.

Your folder now

```
pho-thin/  
  index.html  
  app.js  
  menu.json
```

Watch out From here on you must use Live Server. A page opened as file:// cannot fetch its own folder - the browser blocks it and you get a CORS error.

CORS: Why the Browser Blocks a Fetch

An **origin** is scheme + host + port together - in `http://127.0.0.1:5500/index.html` it is `http://127.0.0.1:5500`. A page may only fetch from its own origin. That rule is **CORS**.

Three steps to hit it

1. You **double-click** `index.html` instead of using Live Server
2. The address bar shows `file:///C:/pho-thin/index.html`
3. `app.js` runs `fetch("menu.json")` - but a `file://` page has origin `null`, matching nothing

Console

```
Access to fetch at
'file:///C:/pho-thin/menu.json' from origin
'null' has been blocked by CORS policy:
Cross origin requests are only supported
for protocol schemes: http, data, https, ...
Failed: TypeError: Failed to fetch
```

Page at <code>http://127.0.0.1:5500</code> fetches	Allowed	Why
<code>menu.json</code>	Yes	Same scheme, host and port
<code>http://127.0.0.1:3000/menu.json</code>	No	Different port
<code>https://api.pho.vn/menu</code>	No	Different host

The fix Your own files: open the folder with Live Server. Someone else's server: only they can allow you, by sending `Access-Control-Allow-Origin` (class 7).

Promises and async / await

- A Promise is a value that is not ready yet. It ends in one of two states: **fulfilled** or **rejected**
- `.then()` schedules what happens **when the value is ready**; `.catch()` handles the failure in one place
- Chaining `.then()` avoids the deeply nested callbacks of old code
- `await` pauses inside an `async` function until the promise settles, then hands you the value
- The result reads top-to-bottom like ordinary code - much easier to follow than a chain of `.then()`
- Wrap it in `try / catch` to handle the rejected case

`await` only works **inside** an `async` function. Writing it anywhere else is a `SyntaxError`.

Example: Promises with .then()

Recall `.then()` schedules what happens when the value is ready, and `.catch()` handles the failure in one place.

app.js

```
fetch("menu.json")
  .then(response => response.json())
  .then(dishes => {
    console.log("how many dishes:", dishes.length);
    console.log("first dish:", dishes[0].name);
    console.log("whole array:", dishes);
  })
  .catch(error =>
    console.log("Failed:", error));
```

Console

```
how many dishes: 3
first dish: Pho bo
whole array:
  (3) [{...}, {...}, {...}]
```

Click the little arrow next to the array in the console to open it and read the three objects.

Example: async / await

Recall `await` pauses inside an `async` function until the promise settles, and `try / catch` handles the rejected case.

app.js

```
async function loadMenu() {  
  try {  
    const res = await fetch("menu.json");  
    const dishes = await res.json();  
    console.log("how many dishes:", dishes.length);  
    console.log("first dish:", dishes[0].name);  
  } catch (error) {  
    console.log("Failed:", error);  
  }  
}  
loadMenu();
```

Console

```
how many dishes: 3  
first dish: Pho bo
```

Same result as the previous slide, same number of lines - but you read it downwards instead of following a chain.

Fetch and Error Handling

- `fetch` is the built-in way to talk to a server - it replaced `XMLHttpRequest` and jQuery's `$.ajax`
- It returns a promise for a `Response`; the body is read separately with `await res.json()`
- `res.ok` is true for status 200-299 and false otherwise; `res.status` is the number
- The promise only rejects when the request itself fails - a badly-formed URL, a blocked request, or a network error
- A 404 or 500 is still a successful request, so it does not reject - you have to check `res.ok` yourself

Watch out This one costs students hours every year: the fetch "worked", the data is missing, and there is no error message - because nobody checked `res.ok`.

Example: Checking res.ok

Recall A 404 or 500 is still a successful request, so fetch does not reject - you must check res.ok yourself.

app.js

```
async function check(file) {  
  const res = await fetch(file);  
  console.log("file:", file, "| ok:", res.ok,  
    "| status:", res.status);  
  if (!res.ok) {  
    throw new Error(`HTTP ${res.status}`);  
  }  
  return await res.json();  
}  
  
check("menu.json")  
  .catch(e => console.log("Caught:", e.message));
```

Console

```
// as written  
file: menu.json | ok: true  
    | status: 200  
  
// now change it to a name  
// that does not exist  
GET ../typo.json  
404 (Not Found)  
file: typo.json | ok: false  
    | status: 404  
Caught: HTTP 404
```

The red GET ... 404 line is printed by the browser, not by your code. It appears even when you handle the error properly.

Example: Putting It Together

Recall Nothing new here: await fetch with res.ok, then for...of and createElement build the list.

app.js

```
const menuList = document.getElementById("menu");
async function loadMenu() {
  try {          // happy path below
    const res = await fetch("menu.json");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const dishes = await res.json();
    menuList.textContent = ""; // clear
    for (const dish of dishes) {
      const li = document.createElement("li");
      li.textContent =
        `${dish.name} - ${dish.price} VND`;
      menuList.append(li);
    }
  } catch (err) { // any failure lands here
    menuList.textContent = "Could not load the menu.";
  }
}
loadMenu();
```

What the page shows

menu.json found

- Pho bo - 60000 VND
- Pho ga - 55000 VND
- Pho tai - 65000 VND

file renamed or server down
Could not load the menu.

Summary

- One folder, three files, one place to paste - that is the whole workspace for today
- JavaScript adds **behaviour**: it turns a page into an application
- Modern JavaScript is **ES6+**: `let/const`, template literals, arrow functions, destructuring
- Objects and arrays carry your data; `map/filter/reduce` transform it without a loop
- The DOM is the page as a tree; `getElementById`, `querySelector` and `addEventListener` read and change it
- Slow work is **asynchronous**: `promises`, `async/await`, and `fetch`
- Always check `res.ok` and handle the failure case - not only the happy path

Class 2 gave you a page that looks right. Today it responds. Class 4 moves the same language to the server.

Practical Work 3: Make Pho Thin Come Alive

Start from your static Pho Thin page from class 2. Today you make it react.

1 - Click a bowl to order

- Give each bowl an `addEventListener("click", ...)`
- On click, add the bowl to a **Your order** list in the DOM
- Show a running **total price** (use `reduce`)

2 - The order form

- A form: customer name + table number
- On submit: `preventDefault`, `validate`, then show a confirmation message

3 - Load today's menu

- fetch the menu from your own `menu.json`
- `async/await`, check `res.ok`, render the bowls into the list
- Handle the failure case with a friendly message

Question for the class Where should each price really be checked - in the browser, on the server, or both? (We answer this in class 7.)

Used an AI assistant? Say what you asked and what you changed.

Practical Work 3: A Starting Point

index.html

```
<ul id="menu">
  <li class="dish"
data-price="60000">Pho bo</li>
  <li class="dish"
data-price="55000">Pho ga</li>
</ul>

<h2>Your order</h2>
<ul id="order"></ul>
<p>Total: <span id="total">0</span>
VND</p>
```

app.js

```
const order = [];
const bowls = document.querySelectorAll("#menu
li");

bowls.forEach(li => {
  li.addEventListener("click", () => {
    order.push(Number(li.dataset.price));
    // TODO: add li.textContent to #order
    const total = order.reduce((s, p) => s +
p, 0);
    document.getElementById("total")
      .textContent = total;
  });
});
```

Try it Finish the TODO: show each ordered bowl's name in the #order list.

Thank you for listening!

Appendix A: Reserved Keywords

Always reserved - never usable as a name

<code>break</code>	<code>do</code>	<code>if</code>	<code>this</code>
<code>case</code>	<code>else</code>	<code>import</code>	<code>throw</code>
<code>catch</code>	<code>enum</code>	<code>in</code>	<code>true</code>
<code>class</code>	<code>export</code>	<code>instanceof</code>	<code>try</code>
<code>const</code>	<code>extends</code>	<code>new</code>	<code>typeof</code>
<code>continue</code>	<code>false</code>	<code>null</code>	<code>var</code>
<code>debugger</code>	<code>finally</code>	<code>return</code>	<code>void</code>
<code>default</code>	<code>for</code>	<code>super</code>	<code>while</code>
<code>delete</code>	<code>function</code>	<code>switch</code>	<code>with</code>

Reserved in strict mode only

<code>implements</code>	<code>interface</code>	<code>let</code>
<code>package</code>	<code>private</code>	<code>protected</code>
<code>public</code>	<code>static</code>	

Reserved only in context

<code>await</code>	<code>// inside modules, async fn</code>
<code>yield</code>	<code>// inside generators</code>

- Modules and class bodies are always strict, so treat the orange list as reserved in any modern file
- `async`, `of`, `get` and `set` are **not** reserved - they are ordinary names that happen to have a special meaning in certain positions

Appendix B: Number Literals

app.js

```
let dec      = 2534;      // decimal
let neg      = -1000;     // negative
let frac     = 0.3555;    // decimals
let exp      = 19.5e-2;   // exponent
let hex      = 0xCCFF;    // hexadecimal
let binary   = 0b1010;    // binary
let octal    = 0o17;      // octal

console.log("exp, hex, bin, oct:",
            exp, hex, binary, octal);
console.log("largest safe int:",
            Number.MAX_SAFE_INTEGER);
console.log("0/0 and 1/0:", 0 / 0, 1 / 0);
```

Console

```
exp, hex, bin, oct:
  0.195 52479 10 15
largest safe int:
  9007199254740991
0/0 and 1/0: NaN Infinity
```

Every literal above is the same type: number. NaN and Infinity are ordinary number values too.

Appendix C: Strings and Escape Sequences

Three ways to quote

```
let single = 'single quotes';  
let double = "double quotes";  
let templ  = `backticks,  
               with ${single}`;  
let empty  = "";
```

Escape sequences

Escape	Character it stands for
<code>\n</code>	New line
<code>\t</code>	Tab
<code>\' \"</code>	Single / double quote
<code>\\</code>	Backslash
<code>\u{1F60A}</code>	Unicode code point

- Single and double quotes behave identically - pick one and stay consistent
- Only backticks allow `${ ... }` and line breaks inside the text

Appendix D: The Other Value Types

app.js

```
let flag      = true;
let nothing = null;
let notSet;

console.log("three types:",
            typeof flag, typeof nothing,
            typeof notSet);
console.log('null vs undefined:',
            null === undefined,
            null == undefined);
console.log("NaN === NaN:",
            NaN === NaN);
```

Console

```
three types: boolean object undefined
null vs undefined: false true
NaN === NaN: false
```

Watch out typeof null returns "object".
It is a bug from 1995 that can never be fixed without breaking the web.

- null = empty on purpose; undefined = never assigned
- NaN is the only value in JavaScript not equal to itself - use `Number.isNaN(x)` to test for it